

Taller de tácticas de disponibilidad — RECAUDO-T

Ping/Echo (detección) + Redundancia Activa (recuperación)

Asignatura: Ingeniería de Software II Repositorio: [taller-disponibilidad](#) Lenguaje: Go 1.21 (solo biblioteca estándar) · React 18 + TypeScript · Docker Compose

1. Contexto y escenario de calidad

RECAUDO-T es el servicio de consulta de saldo de la tarjeta del sistema de transporte masivo. Cada validador de bus consulta el saldo antes de autorizar el ingreso. Una falla de hardware dejó el servicio (instancia única) fuera de línea 11 minutos sin que nadie lo detectara hasta que los usuarios reclamaron.

El rediseño debe lograr dos cosas distintas y complementarias: (a) que una caída se detecte automáticamente y (b) que sea transparente para quien consulta.

Parte del escenario	Valor
Fuente del estímulo	Interna (un proceso servidor deja de responder)
Estímulo	Crash de una réplica
Artefacto	Servicio de consulta de saldo (dispatcher + réplicas)
Entorno	Operación normal, carga constante de validadores
Respuesta	El sistema detecta la falla, la registra en bitácora y sigue atendiendo con las réplicas restantes
Medida de la respuesta	Detección ≤ 3 s y el cliente no percibe errores

Marco conceptual

La disponibilidad se aproxima como:

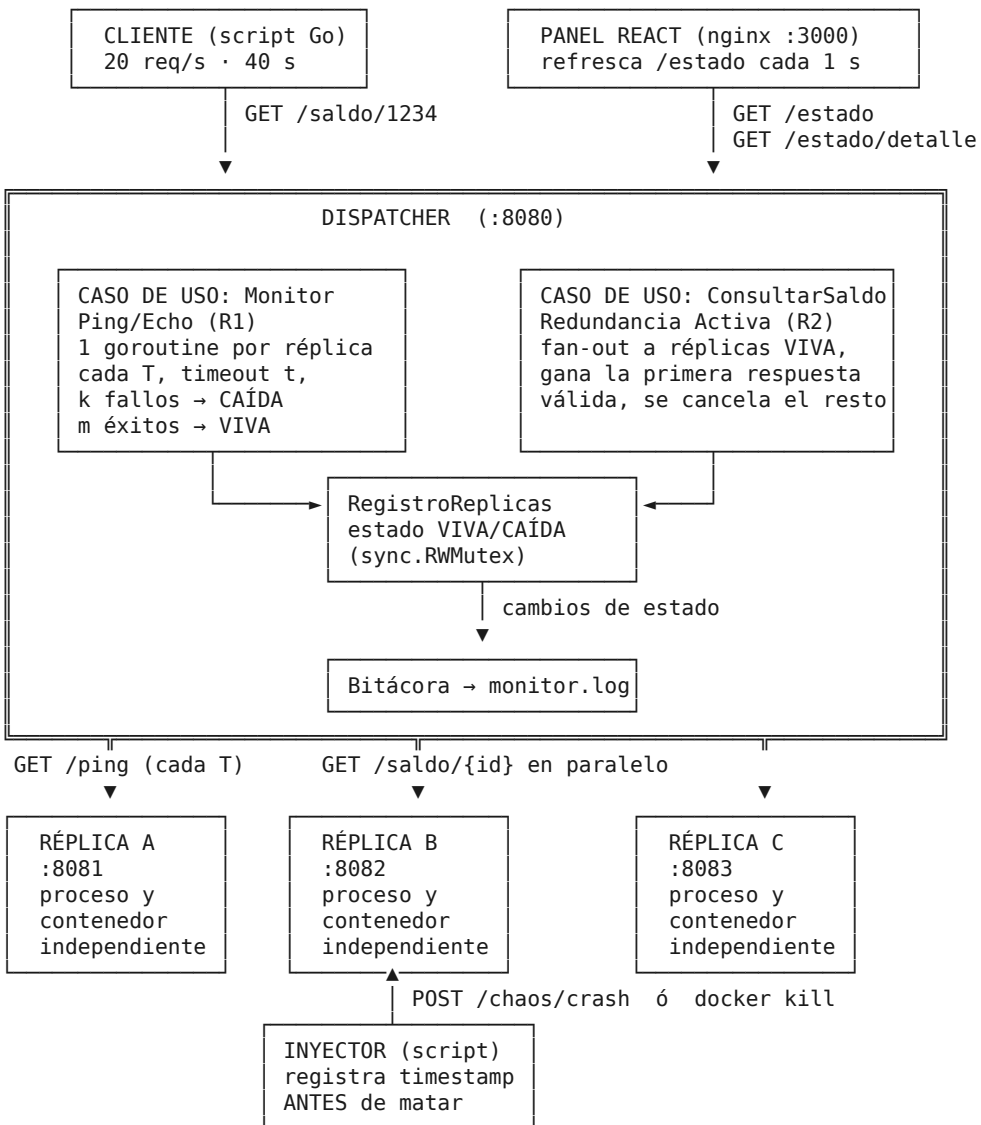
$$A = MTBF / (MTBF + MTTR)$$

Las tácticas implementadas no reducen la frecuencia de fallas (MTBF), sino el tiempo de reparación (MTTR), y por eso vienen en pareja detectar + reaccionar:

- Ping/Echo (detectar). Un monitor sondea periódicamente cada componente y espera un eco dentro de un tiempo límite. Es detección pura: no repara nada.
 - Redundancia Activa (recuperar, *hot spare*). Varias réplicas atienden todas las solicitudes en paralelo y se usa la primera respuesta que llega. Como no hay que "levantar" nada, la recuperación toma milisegundos.
-

2. Arquitectura del sistema

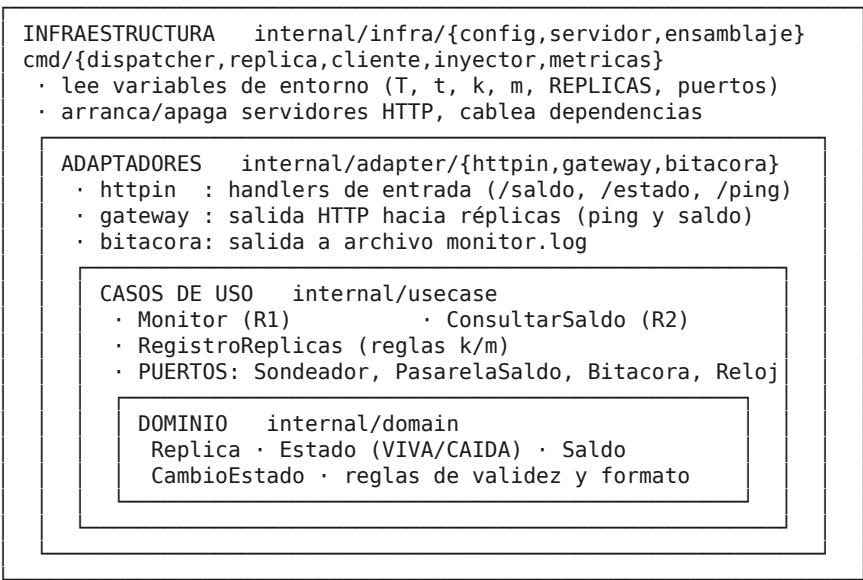
2.1 Diagrama de componentes



Flujo: Cliente → Dispatcher (reenvía en paralelo a las réplicas VIVA) → Réplicas. El monitor sondea en segundo plano, en goroutines independientes que nunca bloquean la atención de consultas.

2.2 Clean Architecture: capas y dependencias

La regla de dependencia se cumple estrictamente: las flechas apuntan siempre hacia adentro; el dominio no conoce HTTP, ni archivos, ni Docker.



Capa	Paquete	Contenido	Depende de
Entidades	<code>internal/domain</code>	<code>Replica</code> , <code>Estado</code> , <code>Saldo</code> , <code>CambioEstado</code> , <code>CalcularSaldo</code> , formato de bitácora	nada (solo stdlib básica)
Casos de uso	<code>internal/usecase</code>	<code>Monitor</code> (R1), <code>ConsultarSaldo</code> (R2), <code>RegistroReplicas</code> , puertos <code>Sondeador</code> / <code>PasarelaSaldo</code> / <code>Bitacora</code> / <code>Reloj</code>	<code>domain</code>
Adaptadores	<code>internal/adapter/httpin</code>	Handlers HTTP del dispatcher y de la réplica	<code>usecase</code> , <code>domain</code>
Adaptadores	<code>internal/adapter/gateway</code>	<code>SondeadorHTTP</code> (ping), <code>PasarelaSaldoHTTP</code> (saldo)	<code>domain</code>
Adaptadores	<code>internal/adapter/bitacora</code>	Archivo: escribe <code>monitor.log</code> + buffer en memoria	<code>domain</code>
Infraestructura	<code>internal/infra/config</code>	Traduce variables de entorno a configuración tipada	<code>domain</code>
Infraestructura	<code>internal/infra/servidor</code>	Arranque y apagado ordenado de HTTP	—
Infraestructura	<code>internal/infra/ensamblaje</code>	Composition root: cablea todo	todas
Binarios	<code>cmd/*</code>	dispatcher, replica, cliente, inyector, metricas	<code>infra</code>

Beneficio concreto de esta separación: las reglas del taller (k fallos para CAÍDA, m éxitos para VIVA, "gana la primera respuesta válida") se prueban sin levantar un solo servidor, con dobles de prueba que implementan los puertos; y las pruebas de integración reutilizan el mismo composition root que el binario de producción (`ensamblaje.ArmarDispatcher`), de modo que lo probado es exactamente lo que corre en el contenedor.

3. Requisitos implementados

3.1 R1 — Ping/Echo (detección)

Requisito del taller	Implementación	Dónde
Sondear <code>GET /ping</code> cada T segundos	Una goroutine con <code>time.Ticker</code> por réplica	<code>usecase/monitor.go:bu</code> <code>cleReplica</code>

Requisito del taller	Implementación	Dónde
Sin bloquear la atención de solicitudes	El monitor corre en goroutines aparte del servidor HTTP; nunca comparten camino de ejecución	ensamblaje + <code>Monitor.Iniciar</code>
Tiempo de espera t; si vence, es fallo	<code>context.WithTimeout(ctx, t)</code> sobre la petición HTTP	<code>usecase/monitor.go:SondearUna</code>
Tras k fallos consecutivos → CAÍDA + bitácora	Contador de fallos consecutivos en el registro; al cruzar k se emite <code>CambioEstado</code> y se escribe	<code>usecase/registro.go:RegistarSondeo</code>
Tras m éxitos consecutivos → VIVA + bitácora	Simétrico al anterior	<code>usecase/registro.go:RegistarSondeo</code>
T, t, k (y m) configurables, no constantes	Variables de entorno <code>MONITOR_T_MS</code> , <code>MONITOR_TIMEOUT_MS</code> , <code>MONITOR_K</code> , <code>MONITOR_M</code>	<code>infra/config + .env</code>
Formato de bitácora	<code>[timestamp] REPLICA_B VIVA -> CAIDA (2 fallos consecutivos de ping)</code>	<code>domain/replica.go:Linea</code>

Detalles de diseño:

- El eco se verifica, no solo el código 200. El sondeo exige 200 y un cuerpo `{"replica_id":"X"}` cuyo identificador coincida con la réplica sondeada. Un 200 vacío o de otro servicio cuenta como fallo.
- Los contadores se reinician al cruzar el umbral y ante el resultado contrario: un éxito intermedio borra los fallos acumulados, tal como exige la palabra "consecutivos".
- Apagado limpio: si el contexto se cancela (SIGTERM de Docker), los fallos provocados por el propio apagado no se contabilizan; así la bitácora no se contamina con falsas caídas.

3.2 R2 — Redundancia Activa (recuperación)

Requisito del taller	Implementación	Dónde
Enviar cada consulta en paralelo a todas las réplicas VIVA	Una goroutine por réplica viva, canal con buffer	<code>usecase/consulta_saldo.go:Ejecutar</code>
Responder con la primera respuesta válida	<code>select</code> sobre el canal de ganadoras; el primero que llega retorna	ídem
Descartar las demás	<code>defer cancelar()</code> cancela el <code>context</code> compartido: las peticiones perdedoras se abortan en la red	ídem
Validez = 200 + saldo disponible	<code>domain.Saldo.EsValida()</code> (replica_id no vacío y saldo ≥ 0)	<code>domain/saldo.go</code>
Número de réplicas configurable, sin tocar el código	Variable <code>REPLICAS="A=http://replica-a:8080,B=..."</code>	<code>infra/config:ParsearReplicas</code>
<code>GET /estado</code> refleja el estado real	Lee el mismo registro que escribe el monitor (RWMutex)	<code>adapter/httpin/dispatcher.go</code>

Detalles de diseño:

- Sin fugas de goroutines: los canales tienen buffer del tamaño de la carrera, de modo que las perdedoras nunca quedan bloqueadas al escribir.
- La carrera está acotada por `CONSULTA_TIMEOUT_MS`; si nadie responde a tiempo, el dispatcher devuelve 503 en vez de colgarse.
- Si no hay réplicas VIVA se devuelve 503 inmediatamente (error de dominio `ErrSinReplicasVivas`), sin intentar tráfico inútil.
- Latencia artificial aleatoria en `/saldo` de cada réplica (`LATENCIA_MIN_MS`–`LATENCIA_MAX_MS`): sin ella, la réplica más cercana ganaría siempre y E0 no evidenciaría que la redundancia compite.

3.3 Restricciones del taller

Restricción	Cumplimiento
"La lógica se implementa, no se importa"	<code>go.mod</code> no tiene una sola dependencia externa: solo <code>net/http</code> , <code>context</code> , <code>sync</code> y <code>encoding/json</code> . No hay Hystrix/Resilience4j ni equivalentes
No usar balanceador con health check incorporado	No hay nginx/HAProxy/Traefik en el camino de datos: el dispatcher es código propio. Tampoco se usan <code>healthcheck:</code> de Docker Compose, para que la única detección sea el monitor del taller
Cada réplica es un proceso independiente	Tres contenedores distintos (<code>recaudo-replica-a/b/c</code>), cada uno con su binario; <code>docker kill</code> mata un proceso real

4. Contrato de la API

Dispatcher (<http://localhost:8080>)

Método y ruta	Respuesta
GET <code>/saldo/{idTarjeta}</code>	200 <code>{"replica_id":"C","id_tarjeta":"1234","saldo":19500,"latencia_ms":7.4,"replicas_consultadas":["A","B","C"]}</code> · 503 si no hay réplicas VIVA o ninguna responde
GET <code>/estado</code>	200 <code>{"A":"VIVA","B":"CAIDA","C":"VIVA"}</code>
GET <code>/estado/detalle</code>	Estado enriquecido (fallos consecutivos, latencia del último ping, configuración T/t/k/m, métricas y bitácora)
GET <code>/bitacora?n=100</code>	<code>{"lineas":["[...] REPLICA_B VIVA -> CAIDA (...)]"}</code>
GET <code>/metricas</code>	<code>{"total":800,"exitosas":800,"fallidas":0,"ganadoras":{"A":301,"B":210,"C":289}}</code>
GET <code>/salud</code>	<code>{"servicio":"dispatcher","estado":"OK"}</code>

Réplica (:8081, :8082, :8083)

Método y ruta	Respuesta
GET <code>/ping</code>	200 <code>{"replica_id":"A"}</code> (responde en < t)
GET <code>/saldo/{idTarjeta}</code>	200 <code>{"replica_id":"A","id_tarjeta":"1234","saldo":19500}</code>
POST <code>/chaos/crash</code>	200 y termina el proceso (código 137). Solo lo usa el inyector
GET <code>/salud</code>	<code>{"servicio":"replica","replica_id":"A","pings":412,"consultas":533}</code>

5. Instalación y ejecución paso a paso

5.1 Requisitos previos

- Docker Desktop 4.x o Docker Engine 24+ con Compose v2 (`docker compose version`).
- (Opcional, solo para correr las pruebas fuera de contenedores) Go 1.21+.
- Puertos libres en el host: `8080`, `8081`, `8082`, `8083`, `3000`.

5.2 Levantar el sistema (un solo comando)

```
git clone <url-del-repositorio>
cd taller-disponibilidad
docker compose up --build
```

Verificación rápida:

```
curl http://localhost:8080/estado      # {"A":"VIVA","B":"VIVA","C":"VIVA"}
curl http://localhost:8080/saldo/1234 # 200 con replica_id y saldo
```

El panel React queda en <http://localhost:3000> y refresca el estado cada segundo.

5.3 Experimento E0 (línea base)

```
.\scripts\experimento-e0.ps1      # Windows
./scripts/experimento-e0.sh       # Linux/macOS/Git Bash
```

Genera 20 req/s durante 40 s → [evidencia/e0_cliente.csv](#).

5.4 Experimento E1 (caída abrupta), dos corridas

```
.\scripts\experimento-e1.ps1 -Replica B -Corrida run1
.\scripts\recuperar-replica.ps1 -Replica B
.\scripts\experimento-e1.ps1 -Replica B -Corrida run2
.\scripts\recuperar-replica.ps1 -Replica B
```

Cada corrida: lanza la carga, espera al segundo 15, ejecuta el inyector (que registra su timestamp antes de **docker kill**) y al terminar muestra bitácora y estado.

5.5 Tabla de métricas

```
./scripts/generar-metricas.sh      # -> evidencia/resultados.md
```

5.6 Pruebas automatizadas

```
./scripts/pruebas.sh              # go vet + unitarias + integración
./scripts/pruebas.sh -race        # con detector de carreras
```

5.7 Detener y limpiar

```
docker compose down              # detiene y elimina los contenedores
docker compose down --rm local   # además elimina las imágenes construidas
```

6. Configuración (variables de entorno)

Ningún parámetro está escrito como constante en el código: todos se leen del entorno en [internal/infra/config](#) y están declarados en [.env](#).

Variable	Defecto	Servicio	Significado
MONITOR_T_MS	1000	dispatcher	T: periodo de sondeo
MONITOR_TIMEOUT_MS	300	dispatcher	t: espera máxima del eco (validado: debe ser < T)
MONITOR_K	2	dispatcher	k: fallos consecutivos para marcar CAÍDA
MONITOR_M	2	dispatcher	m: éxitos consecutivos para volver a VIVA
MONITOR_TRAZA	false	dispatcher	Traza cada ping en stdout (depuración)

Variable	Defecto	Servicio	Significado
ESTADO_INICIAL	VIVA	dispatcher	Estado inicial de las réplicas
REPLICAS	A=http://replica-a:8080,B=...	dispatcher	Lista de réplicas (agregar una es editar esta variable)
CONSULTA_TIMEOUT_MS	1500	dispatcher	Tiempo máximo de la carrera de redundancia
BITACORA_ARCHIVO	/datos/monitor.log	dispatcher	Ruta de la bitácora (volumen ./evidencia)
REPLICA_ID	—	réplica	Identificador único (obligatorio)
LATENCIA_MIN_MS / LATENCIA_MAX_MS	2 / 25	réplica	Latencia artificial de /saldo
SALDO_BASE	15000	réplica	Base del saldo determinista
TASA_RPS	20	cliente	Solicitudes por segundo
DURACION_S	40	cliente	Duración de la carga
ETIQUETA	E0	cliente	Experimento (queda en el CSV)
CSV_SALIDA	/datos/cliente.csv	cliente	Archivo de evidencia
OBJETIVO	recaudo-replica-b	inyector	Contenedor a matar
MODULO_INYECTOR	docker	inyector	docker (kill/stop) o http (/chaos/crash)
PUERTO_DISPATCHER / PUERTO_REPLICA_* / PUERTO_FRONTEND	8080/8081-8083/3000	compose	Puertos publicados en el host

Validaciones de arranque. El dispatcher se niega a arrancar si hay menos de 2 réplicas, si **k** o **m** son menores que 1, si $t \geq T$ o si el puerto es inválido: es preferible fallar rápido a operar con una configuración que invalidaría el experimento.

7. Cómo se miden las métricas

Métrica	Cálculo	Fuente de evidencia
Tiempo de detección	timestamp del cambio VIVA→CAÍDA en la bitácora – timestamp que registró el inyector	evidencia/monitor.log y evidencia/injector.log
% de solicitudes exitosas	filas con exito=1 / total de filas del CSV	evidencia/e*_cliente.csv

Ambos archivos usan el mismo formato de timestamp UTC con milisegundos (2006-01-02T15:04:05.000Z), precisamente para poder restarlos entre archivos. El comando **metricas** (**scripts/generar-metricas.***) automatiza el cálculo y empareja cada inyección con la primera transición VIVA→CAÍDA posterior de la réplica correspondiente.

Valor esperado teórico: con $T = 1$ s y $k = 2$, la detección debe rondar los 2 s (dos ciclos de sondeo), siempre por debajo del límite de 3 s del escenario de calidad.

8. Protocolo experimental y resultados

8.1 Protocolo ejecutado

Parámetro	Valor
Tasa de carga	20 solicitudes/s
Duración de cada corrida	40 s (800 solicitudes)
Tarjeta consultada	1234
Réplicas activas	A, B, C (3 contenedores independientes)
Configuración del monitor	$T = 1000 \text{ ms} \cdot t = 300 \text{ ms} \cdot k = 2 \cdot m = 2$
Momento de la inyección	segundo 15 de la corrida
Réplica sacrificada	B (<code>docker kill recaudo-replica-b</code>)
Corridas	E0 (línea base) + E1 dos veces (run1 y run2)

Máquina de pruebas: Windows 10 Pro + Docker Desktop (Compose v2), con los cinco contenedores en el mismo host.

8.2 Tabla comparativa de métricas

Corrida	Solicitudes	Éxitos	Fallidas	% de éxito	Tiempo de detección	Fallos en los 10 s posteriores a la falla
E0 (línea base)	800	800	0	100,00 %	n/a (sin falla)	n/a
E1 · run1	800	800	0	100,00 %	2,280 s	0
E1 · run2	800	800	0	100,00 %	1,400 s	0

Las dos mediciones de detección quedan por debajo del límite de 3 s del escenario de calidad y son coherentes con el valor teórico $k \cdot T = 2 \text{ s}$: la dispersión corresponde a la fase entre el instante en que muere el proceso y el siguiente tic de sondeo.

Latencia percibida por el cliente (extremo a extremo, a través del dispatcher): $p_{50} \approx 11 \text{ ms}$ y $p_{95} \approx 21\text{-}23 \text{ ms}$, sin diferencia apreciable entre E0 y E1.

8.3 Evidencia — E0 (línea base)

Archivo: `evidencia/e0_cliente.csv` (800 filas).

Reparto de las respuestas ganadoras, que demuestra que la redundancia está compitiendo y no favoreciendo siempre a la misma réplica:

Réplica	Respuestas ganadas	Porcentaje
A	252	31,5 %
B	277	34,6 %
C	271	33,9 %

Fragmento del CSV (columna `replica_id` alternando):

```
secuencia,timestamp_envio,timestamp_respuesta,exito,replica_id,latencia_ms,codigo_http,segundo_relativo,experimento,error
1,2026-09-08T04:04:58.802Z,2026-09-08T04:04:58.811Z,1,A,9.003,200,0.050,E0,
2,2026-09-08T04:04:58.852Z,2026-09-08T04:04:58.856Z,1,A,3.525,200,0.100,E0,
3,2026-09-08T04:04:58.902Z,2026-09-08T04:04:58.913Z,1,C,10.874,200,0.150,E0,
4,2026-09-08T04:04:58.952Z,2026-09-08T04:04:58.965Z,1,B,12.541,200,0.200,E0,
```

Salida del cliente al terminar:


```
[cliente] === RESUMEN E0 ===
[cliente] solicitudes: 800 | exitosas: 800 | fallidas: 0 | % exito: 100.00%
[cliente]   replica A gano 252 carreras (31.5%)
[cliente]   replica B gano 277 carreras (34.6%)
[cliente]   replica C gano 271 carreras (33.9%)
```

8.4 Evidencia — E1 (caída abrupta)

Log del inyector (**evidencia/inyector.log**). El timestamp se escribe antes de matar el contenedor; es el punto de partida de la medición:

```
[2026-09-08T04:06:22.402Z] INYECTOR objetivo=recaudo-replica-b modo=docker comando=kill
[2026-09-08T04:06:22.813Z] INYECTOR resultado=OK duracion_ms=411
[2026-09-08T04:07:02.278Z] INYECTOR objetivo=recaudo-replica-b modo=docker comando=kill
[2026-09-08T04:07:02.576Z] INYECTOR resultado=OK duracion_ms=297
[2026-09-08T04:08:16.177Z] INYECTOR objetivo=recaudo-replica-b modo=docker comando=kill
[2026-09-08T04:08:16.480Z] INYECTOR resultado=OK duracion_ms=302
```

La tercera inyección no pertenece a una corrida con carga: se ejecutó para verificar que el panel React refleja el cambio de estado en vivo. Se conserva porque aporta una medición adicional del tiempo de detección.

Bitácora del monitor (**evidencia/monitor.log**):

```
[2026-09-08T04:06:24.682Z] REPLICA_B VIVA -> CAIDA (2 fallos consecutivos de ping)
[2026-09-08T04:06:38.383Z] REPLICA_B CAIDA -> VIVA (2 exitos consecutivos de ping)
[2026-09-08T04:07:03.678Z] REPLICA_B VIVA -> CAIDA (2 fallos consecutivos de ping)
[2026-09-08T04:07:37.375Z] REPLICA_B CAIDA -> VIVA (2 exitos consecutivos de ping)
[2026-09-08T04:08:17.671Z] REPLICA_B VIVA -> CAIDA (2 fallos consecutivos de ping)
[2026-09-08T04:08:34.369Z] REPLICA_B CAIDA -> VIVA (2 exitos consecutivos de ping)
```

Cálculo del tiempo de detección (bitácora – inyector):

Corrida	t0 (inyector)	t1 (bitácora VIVA→CAÍDA)	Detección = t1 – t0
E1 · run1	04:06:22.402	04:06:24.682	2,280 s
E1 · run2	04:07:02.278	04:07:03.678	1,400 s
Verificación del panel	04:08:16.177	04:08:17.671	1,494 s

Media de las tres mediciones: 1,725 s; máximo 2,280 s. Todas por debajo del límite de 3 s. El cálculo lo reproduce **scripts/generar-metricas.sh**, cuya salida completa está en **evidencia/resultados.md**.

Las líneas **CAIDA -> VIVA** corresponden a la recuperación deliberada de la réplica entre una corrida y la siguiente (**scripts/recuperar-replica.sh B**, que ejecuta **docker start**): también la vuelta a la vida queda registrada, cumpliendo la regla de los m éxitos consecutivos.

`GET /estado` durante la caída, capturado por el script del experimento:

```
{ "A": "VIVA", "B": "CAIDA", "C": "VIVA" }
```

CSV del cliente en la ventana crítica (**evidencia/e1_run1_cliente.csv**, t0 = 04:06:22.402). Todas las filas tienen **exito=1**; obsérvese cómo B deja de ganar carreras justo antes de la falla y A/C absorben todo el tráfico mientras el monitor todavía no se ha enterado:

seq	timestamp_envio	exito	replica	latencia	desfase respecto a t0
684	2026-09-08T04:06:21.946Z	1	B	16.97 ms	t0-0,456 s <- última respuesta ganada por B
693	2026-09-08T04:06:22.397Z	1	A	15.95 ms	t0-0,005 s
694	2026-09-08T04:06:22.446Z	1	C	22.16 ms	t0+0,044 s <- B ya está muerta
695	2026-09-08T04:06:22.497Z	1	A	6.51 ms	t0+0,095 s
...	(43 solicitudes más, todas éxito=1, ganadas por A o C)				
733	2026-09-08T04:06:24.397Z	1	C	23.46 ms	t0+1,995 s
734	2026-09-08T04:06:24.447Z	1	A	13.26 ms	t0+2,045 s
					t0+2,280 s <- el monitor registra VIVA -> CAIDA
739	2026-09-08T04:06:24.697Z	1	C	12.12 ms	t0+2,295 s

Resumen de la ventana: 46 solicitudes se enviaron entre la muerte de B y el instante en que el monitor la marcó CAÍDA, y las 46 tuvieron éxito. Durante esos 2,3 s el dispatcher siguió reenviando también a B (fan-out inútil que fracasaba de inmediato con connection refused), pero como la consulta se envía en paralelo, A y C respondieron en ~10 ms y el cliente nunca percibió nada.

8.5 Análisis de resultados

- 1 La detección funciona y es predecible. 2,280 s, 1,400 s y 1,494 s (media 1,725 s) frente al valor teórico $k \cdot T = 2$ s. La dispersión es la esperada: la falla puede ocurrir en cualquier punto del ciclo de sondeo, de modo que el tiempo de detección se reparte aproximadamente entre $(k-1) \cdot T$ y $k \cdot T$ más la fase inicial. Reducirlo es simplemente bajar T o k, a costa de más tráfico de sondeo y de más falsos positivos ante microcortes de red.
- 2 La redundancia funciona y es transparente. Cero errores en 2 400 solicitudes a lo largo de las tres corridas, y el percentil 95 de latencia no se movió durante la caída.
- 3 Las dos tácticas son independientes. La recuperación (milisegundos) ocurrió unos 2 s antes de la detección. Ninguna sustituye a la otra: la redundancia protege al usuario, el monitor protege al operador.
- 4 La reincorporación también es automática. Tras **docker start**, dos ecos consecutivos bastaron para devolver a B al conjunto de réplicas VIVA; los 13,7 s y 33,7 s transcurridos corresponden a la demora humana en ejecutar el script de recuperación, no al monitor.

9. Pruebas automatizadas

9.1 Cómo ejecutarlas

```
./scripts/pruebas.sh           # go vet + unitarias + integración
./scripts/pruebas.sh -race     # además con el detector de carreras de Go
go test ./... -count=1         # equivalente directo
go test ./tests/... -run TestE1 -v # solo el escenario E1
```

En Windows: **.\scripts\pruebas.ps1** y **.\scripts\pruebas.ps1 -Race**.

Resultado de la corrida de referencia con detector de carreras (**go vet ./... && go test -race ./... -count=1**):

```
ok  github.com/camilin69/taller-disponibilidad/internal/adapter/bitacora 1.556s
ok  github.com/camilin69/taller-disponibilidad/internal/adapter/gateway 1.791s
ok  github.com/camilin69/taller-disponibilidad/internal/adapter/httpin 1.457s
ok  github.com/camilin69/taller-disponibilidad/internal/domain 1.257s
ok  github.com/camilin69/taller-disponibilidad/internal/infra/config 1.384s
ok  github.com/camilin69/taller-disponibilidad/internal/metricas 1.341s
ok  github.com/camilin69/taller-disponibilidad/internal/usecase 2.063s
ok  github.com/camilin69/taller-disponibilidad/tests/integracion 14.009s
```

9.2 Pruebas unitarias (lógica de negocio)

Prueba	Qué garantiza
TestRegistroCaeTrasKFallosConsecutivos	La réplica solo cae al llegar a k fallos, y la transición no se repite
TestRegistroExitoReiniciaContadorDeFallos	"Consecutivos" significa consecutivos: un eco intermedio reinicia el conteo
TestRegistroRevivelTrasMExitos	Regla m: CAÍDA → VIVA tras m ecos seguidos
TestRegistroConcurrenciaSegura	Sondeos y lecturas simultáneas sin carreras de datos (-race)
TestMonitorDetectaCaídaEnTiempoEsperadoYRegistraBitácora	Detección en $\sim k \cdot T$ y formato exacto de la línea de bitácora
TestMonitorTimeoutDeSondeoCuentaComoFallo	Un eco que tarda más que t cuenta como fallo
TestMonitorReviveReplicaTrasMExitos	La recuperación queda registrada en bitácora
TestMonitorReplicaLentaNoBloqueaALasOtras	Una réplica colgada no detiene el sondeo de las demás
TestConsultaDevuelveLaPrimeraRespuestaValida	Gana la más rápida; las lentas no retrasan al cliente
TestConsultaNoEnviaAReplicasCaidas	El fan-out usa solo réplicas VIVA
TestConsultaSobreviveACaídaDeUnaReplica	20/20 consultas exitosas con una réplica muerta todavía marcada VIVA (la situación exacta de E1)
TestConsultaDescartaRespuestasInvalidas	Un 200 sin saldo válido no gana la carrera
TestConsultaSeAgotaElTiempo, TestConsultaSinReplicasVivas, TestConsultaTodasLasReplicasFallan	Errores acotados, sin bloqueos
TestConsultaAlternaGanadorasSegunLatencia	Evidencia de E0: la ganadora alterna
TestConsultaConcurrenteEsSegura	200 consultas simultáneas mientras el monitor escribe el registro
TestParsearReplicas*, TestCargarDispatcher*	T, t, k, m y la lista de réplicas se leen del entorno; validaciones de arranque
TestAgregarReplicaSoloRequiereConfiguracion	Agregar una réplica es configuración, no código
TestCalcularDetecciones*, TestLeerCSVCliente, TestFallosEnVentana	El cálculo de las métricas del informe a partir de la evidencia
TestArchivo* (bitácora)	La bitácora persiste en disco, abre en modo append y es segura en concurrencia
TestSondeador* (gateway)	El eco debe ser 200, legible y con el identificador correcto; respeta el tiempo límite t
TestPasarela* (gateway)	Solo un 200 con cuerpo válido cuenta como respuesta de una réplica
TestDispatcher* (handlers)	Contrato de /estado, 503 sin réplicas VIVA, validación del id de tarjeta, CORS
TestReplicaSaldoEsDeterministaEntreReplicas	Todas las réplicas devuelven el mismo saldo para la misma tarjeta

9.3 Pruebas de integración (sistema completo)

Levantamos réplicas HTTP reales y el dispatcher real usando el mismo **composition root** del binario de producción (*ensamblaje.ArmadorDispatcher*), con bitácora en archivo. No requieren Docker.

Prueba	Qué reproduce
TestE0LineaBaseSinFallas	E0: carga a 20 req/s, 100 % de éxito y más de una réplica respondiendo
TestE1CaídaAbruptaDeteccionYTransparencia	E1 completo: mata una réplica en caliente y verifica (a) detección ≤ 3 s + línea en <i>monitor.log</i> + /estado actualizado y (b) cero errores del cliente
TestReplicaSeRecuperaYVuelveAViva	Reincorporación tras m ecos, registrada en bitácora
TestSinReplicasVivasElDispatcherResponde503	Con todas las réplicas caídas el dispatcher responde 503 y no se cuelga

Prueba	Qué reproduce
TestContratoDeEstado	GET /estado devuelve {"A": "VIVA", ...}
TestAgregarUnaCuartaReplicaSinTocarCodigo	Cuatro réplicas funcionando solo por configuración
TestGanchoDeCaosDeLaReplica	POST /chaos/crash responde 200 y solicita terminar el proceso

Salida de la corrida de referencia (go test ./tests/... -run 'TestE0|TestE1' -v):

```
sistema_test.go:173: E0: 60 solicitudes, 100% exito, reparto map[A:22 B:17 C:21]
--- PASS: TestE0LineaBaseSinFallas (3.03s)
sistema_test.go:198: replica B terminada a las 2026-09-07T22:54:21.665-05:00
sistema_test.go:211: tiempo de deteccion medido: 1.317 s
sistema_test.go:240: E1: 120 solicitudes, 120 exitosas (100%), reparto map[A:52 B:11 C:57]
--- PASS: TestE1CaidaAbruptaDeteccionYTransparencia (7.23s)
PASS
```

10. Preguntas de análisis

10.1 En E1, ¿el cliente dejó de recibir respuesta antes o después de que el monitor detectara la caída? ¿Por qué ocurre eso y qué dice sobre la relación entre Ping/Echo y redundancia activa?

Ni antes ni después: el cliente nunca dejó de recibir respuesta. En la corrida run1, la réplica B murió en $t_0 = 04:06:22.402$ y el monitor la marcó CAÍDA en $t_0 + 2,280$ s. Entre esos dos instantes el cliente envió 46 solicitudes y las 46 tuvieron éxito (**exito=1**), ganadas por A o por C, con latencias de ~10 ms.

Ocorre porque las dos tácticas trabajan en escalas de tiempo distintas: la consulta se envía simultáneamente a todas las réplicas marcadas VIVA, así que la desaparición de una de ellas queda "reparada" en el tiempo que tarda otra réplica en contestar (milisegundos), mientras que el monitor necesita acumular k fallos espaciados T segundos (≈ 2 s) para saber que la réplica murió. Durante esa ventana el dispatcher siguió mandando peticiones a un proceso muerto —tráfico desperdiciado que fracasaba de inmediato con connection refused—, pero eso no afectó al cliente porque la respuesta válida ya venía en camino desde otra réplica.

La conclusión es la del marco conceptual: Ping/Echo detecta pero no repara, y la redundancia activa repara pero no sabe. Ping/Echo reduce el tiempo de diagnóstico (de 11 minutos a 2 segundos) y la redundancia activa reduce el tiempo de reparación percibida (a prácticamente cero). Son complementarias y ninguna sustituye a la otra.

10.2 Si solo hubiera implementado Ping/Echo (sin redundancia), ¿el usuario habría notado la falla igual? ¿Y si solo hubiera implementado redundancia, sin monitor? ¿Para qué serviría entonces el monitor?

Solo Ping/Echo: el usuario la habría notado exactamente igual. Con una sola instancia, al morir el proceso todas las consultas fallan; el monitor solo habría conseguido que alguien se enterara a los 2 s en lugar de a los 11 minutos. Eso reduce el MTTR —la reparación empieza antes— pero no lo elimina: la indisponibilidad percibida sigue existiendo, solo que ahora dura lo que tarde la reparación y no lo que tarde el primer reclamo. En $A = \text{MTBF}/(\text{MTBF} + \text{MTTR})$ mejora el MTTR, pero la falla no se vuelve transparente.

Solo redundancia, sin monitor: el usuario no habría notado nada... por un tiempo. El problema es que tampoco lo habría notado el operador. Consecuencias concretas:

- Las réplicas irían muriendo silenciosamente, una a una, hasta que caiga la última: la falla que finalmente se ve es la total, y llega sin aviso previo.
- El sistema operaría con una redundancia real menor que la que cree tener (tres réplicas en el papel, una en la práctica) y nadie podría saberlo.
- El dispatcher seguiría enviando tráfico a procesos muertos indefinidamente, desperdiciando conexiones y arriesgando latencia: si el proceso no rechazara la conexión sino que la dejara colgada, cada consulta dependería del timeout.
- No habría métricas para dimensionar capacidad ni evidencia para el postmortem.

Es decir: el monitor sirve al operador, no al usuario. Aporta observabilidad (`GET /estado`, bitácora), permite alertar y reparar antes de que se agote la redundancia y —en este diseño— además optimiza el camino de datos, porque el dispatcher deja de enviar consultas a las réplicas que sabe caídas.

10.3 Si una réplica responde a `/ping` con normalidad pero devuelve un saldo incorrecto, ¿su monitor lo detectaría? ¿Por qué no, y qué táctica adicional del árbol de disponibilidad ayudaría?

No lo detectaría. El monitor implementado comprueba únicamente vivacidad: que exista un proceso que conteste `200 {"replica_id":"B"}` dentro de t milisegundos. `/ping` ni siquiera toca los datos de saldo, así que una réplica con la base de datos desactualizada, un error de cálculo o memoria corrupta seguiría siendo, para el monitor, perfectamente sana. Peor todavía: como en la redundancia activa "gana la primera respuesta", si esa réplica corrupta resulta ser la más rápida, el cliente recibiría el saldo incorrecto. Es un fallo bizantino (respuesta equivocada) frente al fallo por parada (fail-stop) que este diseño sí cubre.

Tácticas adicionales del árbol de disponibilidad que ayudarían, todas de la rama detectar fallas:

- Votación (`*voting*`) / comparación de réplicas: consultar a las tres y responder con el valor en el que coincida la mayoría (2 de 3), marcando como sospechosa a la que disiente. Es la contrapartida natural de la redundancia activa cuando el fallo puede ser bizantino.
- Monitoreo de condición (`*condition monitoring*`) y chequeo de sensatez (`*sanity checking*`): validar invariantes del dominio antes de responder —saldo no negativo, no superior al tope de la tarjeta, sin variaciones imposibles respecto de la última lectura conocida.
- Autoprueba (`*self-test*`) y sumas de verificación (`*checksum*`): que cada réplica verifique periódicamente la integridad de sus datos y se autoexcluya si la comprobación falla; complementariamente, resincronización de estado para reintegrarla una vez corregida.

La mejora más barata sería un ping semántico: en lugar de un eco vacío, consultar una tarjeta testigo de saldo conocido, lo que ya detectaría una parte de estos fallos.

10.4 Con disponibilidad individual de 0,98 y fallas independientes, calcule la disponibilidad del sistema con 2 y 3 réplicas. ¿En qué caso real ese supuesto de independencia sería falso?

Con réplicas en paralelo basta que una funcione, de modo que la indisponibilidad conjunta es el producto de las indisponibilidades individuales:

$$A(n) = 1 - (1 - p)^n, \text{ con } p = 0,98 \rightarrow (1 - p) = 0,02$$

Réplicas	Cálculo	Disponibilidad	Indisponibilidad anual
1	0,98	0,98 (98 %)	≈ 175,2 h/año (7,3 días)

Réplicas	Cálculo	Disponibilidad	Indisponibilidad anual
2	$1 - 0,02^2 = 1 - 0,0004$	0,9996 (99,96 %)	≈ 3,5 h/año
3	$1 - 0,02^3 = 1 - 0,000008$	0,999992 (99,9992 %)	≈ 4,2 min/año

Cada réplica adicional divide entre 50 la indisponibilidad: el salto de 1 a 2 réplicas es enorme y el de 2 a 3 sigue siendo grande, pero los rendimientos decrecientes aparecen rápido frente al costo de operar una réplica más.

Cuándo el supuesto de independencia es falso (fallas de causa común):

- Mismo sustrato físico: las tres réplicas en el mismo servidor, rack, fuente de poder, switch o zona de disponibilidad. Es exactamente el caso de este taller: los tres contenedores corren sobre el mismo demonio Docker y la misma máquina, así que apagar el equipo las tumba a las tres a la vez y la disponibilidad real es la del host, no 0,999992.
- Mismo software y mismos datos: todas ejecutan la misma imagen, de modo que un error determinista (un desbordamiento con cierta tarjeta, una fuga de memoria, un certificado vencido) las tumba simultáneamente. Lo mismo si comparten una única base de datos: esa dependencia común es el verdadero punto único.
- Misma operación: un despliegue o un cambio de configuración equivocado aplicado a las tres al mismo tiempo.
- Carga correlacionada: cuando una réplica cae, su tráfico se reparte entre las restantes; si estaban al límite, la sobrecarga provoca un efecto dominó.

En la práctica esto se modela con un factor de causa común (modelo beta): $A_{real} = 1 - [\beta \cdot (1-p) + (1-\beta) \cdot (1-p)^n]$. Con apenas un 5 % de fallas correlacionadas, tres réplicas dejan de valer 0,999992 y caen a ≈ 0,999.

10.5 El dispatcher es ahora el único punto por el que circula todo el tráfico. ¿Introdujo un nuevo punto único de falla al resolver el anterior?

Sí, formalmente sí. El dispatcher está en serie con el conjunto redundante, de modo que la disponibilidad total es un producto:

$$A_{sistema} = A_{dispatcher} \times [1 - (1 - p)^n]$$

Con un dispatcher de 0,98 y tres réplicas, el sistema no supera 0,98: toda la mejora de las réplicas queda limitada por el eslabón en serie. Redundar solo la capa de atrás y dejar única la de adelante es un error clásico de diseño.

Ahora bien, el punto único que se introdujo es cualitativamente mejor que el que se eliminó, y eso es lo que hace razonable el diseño:

- El servicio original era con estado y lento de reparar: guardaba los saldos, y su caída significó 11 minutos de indisponibilidad más el tiempo de recuperación de los datos.
- El dispatcher es sin estado: su única memoria es el estado VIVA/CAÍDA, que no se persiste y se reconstruye solo sondeando, es decir en $k \cdot T \approx 2$ s. Se puede reiniciar, mover o clonar sin coordinación, y su MTTR es de milisegundos.

Mitigaciones estándar, en orden de costo:

- 1 Reinicio automático supervisado (**restart: always** en Compose, o un supervisor tipo systemd/Kubernetes): reduce el MTTR del dispatcher a segundos. En este taller se dejó **restart: "no"** a propósito, para que la réplica que mata el inyector permanezca muerta y el experimento sea observable.
- 2 Varios dispatchers en paralelo detrás de una IP virtual o DNS round-robin y, mejor aún, redundancia del lado del cliente: que el validador conozca dos o tres URLs de dispatcher y aplique la misma lógica de "primera respuesta válida". Como el dispatcher no comparte estado, escalarlo es trivial.
- 3 Eliminar el intermediario: llevar el fan-out al propio cliente (una biblioteca dentro del validador). Es lo que más disponibilidad ofrece y lo que más difícil hace operar y actualizar la política de disponibilidad.

En resumen: el nuevo punto único es real y hay que reconocerlo, pero se cambió un componente con estado y MTTR de minutos por uno sin estado con MTTR de milisegundos y trivialmente replicable. La tarea pendiente para producción es el punto 2.

11. Decisiones tomadas ante ambigüedades del enunciado

Ambigüedad	Decisión	Justificación
El enunciado fija k y menciona m sin precisar su relación	k y m independientes (MONITOR_K , MONITOR_M), con m = k = 2 por defecto	Permite histéresis asimétrica (caer rápido, revivir con más evidencia) sin tocar el código
Estado inicial de las réplicas	VIVA por defecto, configurable con ESTADO_INICIAL	El sistema queda operativo desde el primer segundo; si una réplica no arrancó, el monitor la marca CAÍDA en k-T
"Respuesta válida (200 + saldo)"	Se exige 200 y cuerpo con replica_id no vacío y saldo ≥ 0	Un 200 vacío no debe ganar la carrera
¿Todas las réplicas devuelven el mismo saldo?	Sí: el saldo es determinista por tarjeta (base + hash(idTarjeta) mod 50 × 100)	Son réplicas del mismo dato, no servicios distintos
El taller no dice cómo evitar que gane siempre la misma réplica	Latencia artificial aleatoria en /saldo (LATENCIA_MIN_MS - LATENCIA_MAX_MS)	Sin ella, E0 no evidenciaría que la redundancia compite
Cómo mata el inyector a la réplica	Dos modos: docker kill (predeterminado, más fiel a un crash) y POST /chaos/crash	El primero no requiere que el proceso colabore; el segundo permite el experimento sin Docker
Uso de healthcheck : de Docker Compose	No se usa	La restricción prohíbe resolver la detección con mecanismos ya hechos: la única detección es el monitor propio
Punto de partida del tiempo de detección	El timestamp que el inyector escribe antes de invocar docker kill	Es la medida conservadora: incluye el tiempo del propio comando ($\approx 0,3$ s)
Puertos del host ocupados	Todos los puertos publicados son variables (PUERTO_DISPATCHER , PUERTO_REPLICA_* , PUERTO_FRONTEND)	Permite correr el taller en máquinas con otros servicios levantados sin tocar el compose

12. Limitaciones conocidas y trabajo futuro

- Fallos bizantinos no cubiertos (respuesta rápida pero incorrecta): véase la respuesta 10.3; requeriría votación y chequeos de sensatez.
- El dispatcher sigue siendo un punto único (véase 10.5): faltaría un segundo dispatcher y redundancia del lado del cliente.
- Las tres réplicas comparten host: la independencia estadística que supone el cálculo de 10.4 no se cumple en el entorno de laboratorio.

- La bitácora no rota: para una operación prolongada convendría rotación por tamaño o por fecha.
 - El estado del monitor vive en memoria: al reiniciar el dispatcher se pierde el histórico en memoria (el archivo `monitor.log` sí persiste) y el estado se reconstruye por sondeo en $k \cdot T$.
-

13. Conclusiones

- 1 Se implementaron y se midieron las dos tácticas exigidas: Ping/Echo detectó la caída de una réplica en 2,280 s, 1,400 s y 1,494 s en tres inyecciones independientes, siempre por debajo del límite de 3 s del escenario de calidad y dejando registro persistente y verificable en la bitácora.
- 2 La redundancia activa hizo la falla completamente transparente: 100 % de solicitudes exitosas en las tres corridas (2 400 solicitudes), incluidas las 46 que se enviaron mientras el sistema todavía no sabía que la réplica había muerto.
- 3 Las mediciones confirman experimentalmente la tesis del marco conceptual: detección y recuperación son problemas distintos, con escalas de tiempo distintas, y una arquitectura disponible necesita las dos.
- 4 La separación en capas (Clean Architecture) permitió probar las reglas de negocio —umbrales k/m , carrera de redundancia— sin levantar infraestructura, y reutilizar el mismo cableado de producción en las pruebas de integración.